Haipipe-page: ownership and workflow

1 Page ownership

Division map: editable sources and generated delivery in one Page Folder.

```
Page Folder
+-- page.toml         source registration
+-- Page.md           Opening . Content . Aims
+-- draft/          plan . context . evidence
+-- delivery/web/     generated reading site
```

A Page Folder keeps everything needed to understand, change, and deliver one Page in one place. It can hold the Markdown source, code, figures, evidence, Runs and Results, and generated web, LaTeX, and Word files. Studio is where people brainstorm for the Page. Studio Chat keeps the conversation, while Studio Draw turns ideas into visual sketches. The Markdown source uses the same stem as the folder: haipipe-page-guide/ corresponds to haipipe-page-guide.md. Each delivery file is generated from that Markdown source instead of maintained as a separate source. The earlier HTML draft is kept only as material and does not control this Page or its outputs.

2 Board and workbenches

Division map: Board navigation and workbench workspaces share the Page's sources.

```
Board membership --> Page Face
                       |
                  Page Folder
                  +-- Outline   planning and evidence
                  +-- Studio    chat and drawing
                  +-- Runs
                  |   +-- Page Run   rpNN . feedback . Versions . Steps
                  |   +-- Task Run   native id . delegated output
                  +-- Delivery  generated outputs
                  +-- Folder    file inventory
```

Board supplies grouping, ordering, and navigation by registering the same Page Face. Removing that membership should leave the Page Folder and its content intact. Workbenches present the Page's planning records, conversations, runs, outputs, and files through their own declared writers. A workbench is not enabled merely because a folder or a button bears its name; its surface must reach the real records and supported actions.

A Page Run has a local identity and a Version that stores its feedback Steps. Task Runs keep their native identity and return delegated outputs, such as paragraph drafts or Discovery results, for the Page to use. The `rpNN` and Task `rNN` sequences are independent, so `rp01` and `r01` may coexist in one Page Folder. An automated CONTEXT-to-CHECK controller invocation is a Page workflow pass, not another Page Run.

The Page's Runs function proposes bounded interaction goals when a person needs to shape, compare, revise, or accept the Page itself. A proposal is not yet a Run and receives no `rpNN` until the person selects or directly commissions it. A selected candidate resumes a matching open Page Run or allocates the next `rpNN` for an independent goal, and later feedback appends Steps to that Run's current Version. Code, search, data, rendering, build, Discovery, and other output-producing work remains a normal Task Run even when a later human gate reviews its Result.

The first Page Run is always `rp-struct-01`: one shared Structure Run in which the person and agent complete both SHAPE and SURVEY for the whole Page. The Structure Run's closure freezes the Page-global reading order as `P01`, `P02`, through `PN`, with every serial mapped to its plan address; multiple people may contribute Steps to this same Run. After `rp-struct-01` closes, the Runs function creates paragraph Runs, each with a scope covering one or more of the `N` numbered paragraphs. Each selected paragraph Run uses a typed identity that exposes the exact serial or contiguous range, such as `rp-para-01_P01` or `rp-para-02_P02-P03`; its descriptive wording stays in Goal. Different human questions or acceptance boundaries call for different Step scopes, while code, Discovery, data, rendering, and build remain normal Task Runs.

```
flowchart LR
  O["rp-struct-01"] -->|human closes SHAPE + SURVEY + P01..PN| I["numbered paragraph index"]
  I --> R1["rp-para-01_P01"]
  I --> R2["rp-para-02_P02-P03"]
  I --> RN["rp-para-NN_PNN"]
```

3 Continuing work

Division map: feedback changes a bounded source before delivery is refreshed.

```
Your request

Context and current plan

Candidate change + feedback record

Explicit scoped acceptance

Adopted Content  refreshed delivery
```

A request to revise Opening sets the editing boundary; it does not authorize rewriting the rest of the Page. During planning, the Outline workspace pairs each planned point with candidate prose for discussion. A Writing Run preserves the goal and feedback history through Versions and Steps, while accepted wording is adopted into Content through the Page workflow. Those records do not imply that a browser button executes an agent, and accepting one passage does not accept the whole Page.